SECURITY ENGINEERING | FIELD REFERENCE

WEB PENETRATION TESTING

50 Essential Test Cases Based on OWASP Top 10

A practical field guide for testing any website — written for security engineers, bug bounty hunters, and developers who want to understand how attackers think and how to find what they find.

Written by

Yamini Yadav

Security Engineer

SQLi · XSS · JWT · IDOR · SSRF · XXE · Auth · Business Logic · Deserialization

SECURITY ENGINEERING | FIELD REFERENCE

About This Guide

My name is Yamini Yadav and I am a Security Engineer. I wrote this guide because I could not find a single resource that explained web application security testing the way I needed it when I was learning — in plain, honest language that actually makes sense to a human being, not just someone who already knows everything.

Every test case in this book is something I have personally run against real applications during security assessments and bug bounty engagements. The descriptions of what each vulnerability is and how to find it are written the way I would explain them to a colleague sitting next to me — not as abstract theory, but as the actual thought process you go through when you are looking at a live application and trying to understand whether it is secure.

This is not a beginners-only book and it is not an experts-only book. It is a field reference — the kind of document you keep open in a second window while you are actively testing a target. Each test case stands on its own. You do not need to read from the beginning. Jump to whichever category is relevant to your current engagement and follow the step-by-step testing guide.

What This Guide Covers

All 50 test cases are organized around the OWASP Top 10 framework — the industry-standard classification of the most critical web application security risks. The categories covered include injection attacks of all types, broken authentication including JWT vulnerabilities, sensitive data exposure, XML External Entity injection, broken access control including IDOR and privilege escalation, security misconfigurations, Cross-Site Scripting in all three forms, insecure deserialization, vulnerable and outdated components, server-side request forgery, and business logic vulnerabilities.

How to Use This Book

Each test case card has two main sections. The first — What is this vulnerability — explains it in plain English so you understand what you are actually looking for and why it matters. The second — How to find it, step by step — gives you a concrete, actionable testing methodology you can follow immediately on any target.

There is no severity badge on the test cards intentionally. Every vulnerability matters in the right context. A medium finding on an admin login page can be more impactful than a high finding on a static marketing page. Learn to assess impact based on context, not just a color label.

> ***"Security is not a product you buy. It is a process you practice every single day."***
>
> — Yamini Yadav, Security Engineer

Table of Contents

Category: Injection

TC-01 SQL Injection (Classic)

WHAT IS THIS VULNERABILITY

Imagine the login page of a website. When you type your username and password, the website builds a small database question behind the scenes that looks like: give me the user whose username is john and password is mypassword. That question goes to the database and if a match is found, you are logged in. SQL Injection happens when an attacker types something clever instead of a real username. Something like: ' OR '1'='1. Suddenly that database question turns into: give me the user whose username is anything OR where 1 equals 1. And since 1 always equals 1, the database returns every single user and the attacker gets in without a real password. This is one of the oldest bugs in web security and it is still found everywhere today because developers forget to treat user input as untrusted data that must never be executed as code.

HOW TO FIND IT — STEP BY STEP

Start by identifying every place where the application takes user input — login fields, search bars, URL parameters like ?id=5, dropdown filters, and contact forms. The simplest test is putting a single quote character ' into any field and submitting. If the page crashes, shows a database error, or behaves differently than usual — a potential injection point is found. Go deeper by trying: ' OR '1'='1'-- in the username field with any password. If you get logged in without a real password, SQL injection is confirmed. For URL parameters like ?id=5 try changing it to ?id=5' and observe the response closely. Then hand the confirmed endpoint to SQLmap and let it automate the full extraction — it identifies the database type, extracts table names, column names, and dumps the data to show the real impact.

TOOLS TO USE

SQLmap | Burp Suite | OWASP ZAP

TC-02 Blind SQL Injection

WHAT IS THIS VULNERABILITY

Blind SQL Injection is the sneaky cousin of classic SQLi. The vulnerability is exactly the same — the database is accepting attacker-controlled commands — but this time the application does not show any error messages or raw data. It just shows a normal page. So how do you know it is working? You ask the database True or False questions and watch how the page behaves differently. If the page shows different content when the answer is True versus when it is False, you can slowly extract every password character by character. There is also a time-based version: you make the database pause for exactly 5 seconds on purpose. If the page takes 5 extra seconds to load after your payload, injection is confirmed even when you cannot see any database output at all.

HOW TO FIND IT — STEP BY STEP

Pick any input field and send two payloads back to back. First: ' AND 1=1-- which should be True — note what the page looks like. Then: ' AND 1=2-- which should be False — compare the two responses carefully. Even a small difference in word count, a missing element, or a slightly different layout confirms blind injection. For time-based testing inject: ' AND SLEEP(5)-- on MySQL targets. Time the server response in Burp Suite using the built-in response timer. If the server takes exactly 5 extra seconds to reply, blind SQLi is confirmed without seeing a single byte of output. From here use SQLmap with --technique=BT and flags --level 5 --risk 3 to automate the full character-by-character data extraction.

TOOLS TO USE

SQLmap | Burp Suite Intruder

TC-03 NoSQL Injection

WHAT IS THIS VULNERABILITY

NoSQL databases like MongoDB do not use SQL — they use JSON-style queries. So traditional SQL payloads do not work on them. But they have their own injection problem. MongoDB has special operators like $gt meaning greater than, $ne meaning not equal, and $regex meaning match this pattern. If an application passes user input directly into a MongoDB query without cleaning it first, an attacker can inject these operators as part of the request body. The classic attack targets a login form. Instead of typing a real password, the attacker sends: password: {$gt: empty string}. MongoDB reads this as: find a user whose password is greater than an empty string — which matches literally every user in the database. The first one returned is whoever gets logged in, often the admin account.

HOW TO FIND IT — STEP BY STEP

Look for applications that accept JSON in POST request bodies — common in modern REST APIs and single-page apps. In Burp Suite, intercept a login request and check if Content-Type is application/json. Then modify the password value. Change password: mypassword to password: {"$gt": ""}. Resend the request. If you receive a successful login response without the real password, NoSQL injection is confirmed. Also try $ne: null and $regex: .* as alternative payloads. For form-based endpoints try appending: username[$ne]=nonexistent&password[$ne]=nonexistent in the request body. The tool NoSQLMap automates detection and exploitation across MongoDB, CouchDB, Redis, and other NoSQL backends.

TOOLS TO USE

Burp Suite | NoSQLMap | manual testing

TC-04 Command Injection

WHAT IS THIS VULNERABILITY

Some web application features talk directly to the server operating system. Think of a website feature that lets you ping an IP address and shows you the result, or converts a file format, or sends a test email. Under the hood the developer wrote code that takes your input and builds a shell command from it — something like: run ping and then whatever IP address you typed. Command injection happens when an attacker adds extra shell commands using separator characters. The semicolon separates commands in bash. So if you type 192.168.1.1; whoami, the server runs ping AND ALSO runs whoami. If the output comes back to you, you now know the username the web server runs as. From there the attacker can read files, create new users, download malicious tools, or take complete control of the server.

HOW TO FIND IT — STEP BY STEP

Look for any feature that seems to interact with the operating system — network utilities like ping, file converters, image processors, backup generators, or anything that takes input and produces an output file. In Burp Suite capture the request and try adding these payloads right after the legitimate input: semicolon followed by sleep 5 to test with a delay, then pipe followed by whoami, then double ampersand followed by id. Watch both the response time and content. If the page takes 5 extra seconds after the sleep payload, injection is working even without visible output. If you see the result of whoami or id anywhere in the response, Remote Code Execution is confirmed. Use the tool Commix which tests dozens of injection techniques and separator combinations automatically.

TOOLS TO USE

Burp Suite | Commix | manual payload testing

TC-05 LDAP Injection

WHAT IS THIS VULNERABILITY

LDAP is the technology that large companies use to manage employee accounts — it is what powers corporate logins and Active Directory. When a web application authenticates users against an LDAP directory, it builds a query that looks for your username in the directory tree. If that query is built using raw user input without sanitization, an attacker can inject LDAP filter syntax to change its meaning. The classic attack injects characters that close the current filter and open a new one that always evaluates to true — bypassing the password check completely. This is especially dangerous in corporate environments where the LDAP directory contains sensitive employee information, organizational structure, email addresses, and phone numbers.

HOW TO FIND IT — STEP BY STEP

Target login forms on applications connected to enterprise directories — internal corporate tools, VPN portals, HR systems, or university login pages. In the username field try payloads like: admin)(&) or *)(uid=*))(|(uid=* or)(|(password=* with any password. Submit and observe the response. If you get logged in without a valid password, or if more data comes back than expected, LDAP injection is confirmed. Because LDAP errors are often hidden by the application, also use Burp Suite's response comparison feature — send a normal attempt then the injection attempt and diff the two responses byte by byte. Any difference in response size or content structure is worth investigating further.

TOOLS TO USE

Burp Suite | OWASP ZAP | manual testing

TC-06 XPath Injection

WHAT IS THIS VULNERABILITY

XPath Injection targets applications that store data in XML files and use XPath queries to search them. Some older enterprise systems store user accounts and configuration in XML rather than databases. When a login form queries an XML file using XPath, the query might look for a user node where the username equals what you typed AND the password equals what you typed. If the input is not sanitized, an attacker can inject XPath syntax that creates an always-true condition. The sequence ' or '1'='1 closes the current string comparison and adds an OR condition that is always true — so the password check is completely skipped and the attacker is logged in as the first user in the XML file, which is usually the administrator.

HOW TO FIND IT — STEP BY STEP

Test any login page by entering ' or '1'='1 as both the username and password simultaneously. If you get logged in without real credentials, XPath injection is likely. To distinguish from SQL injection, also try the classic SQL payload ' OR 1=1-- — if the SQL one fails but the XPath one works, you have specifically confirmed XPath injection. Next test boolean extraction: ' or '1'='1' or 'a'='a should always be true, versus ' or '1'='2' or 'a'='b should always be false. If responses differ, you can use these boolean conditions to extract the entire XML file contents character by character, similar to blind SQL injection methodology.

TOOLS TO USE

Burp Suite | manual testing

TC-07 HTML Injection

WHAT IS THIS VULNERABILITY

HTML Injection is where an attacker injects raw HTML markup into a page and the browser renders it. Unlike XSS which injects JavaScript, HTML injection injects markup like headings, links, images, and forms. While this sounds less severe, it is dangerous for phishing. An attacker can inject a fake login form into a legitimate trusted page. The user sees the real website URL in their browser bar and trusts the page completely — but they are actually looking at attacker-controlled content overlaid on the real page. They type their credentials into the fake form and submit it to the attacker's server. The user never realizes they were tricked because the URL showed their bank or employer's real domain the entire time.

HOW TO FIND IT — STEP BY STEP

Go to any input field on the application — search boxes, comment fields, profile name fields, feedback forms, or anywhere your input gets shown back to you later. Type: <h1>Testing HTML Injection</h1> and submit. Then navigate to the page where that text should appear. If you see a large formatted heading instead of the raw text being displayed, the application is rendering your HTML tags. To escalate, try injecting a fake login form pointing to an external URL. If that form renders on the legitimate page as a real form, take a screenshot immediately — this is a high-quality phishing attack vector that deserves a separate detailed report entry.

TOOLS TO USE

Burp Suite | manual browser testing | OWASP ZAP

Category: Broken Auth

TC-08 Broken Authentication: Weak Password Policy

WHAT IS THIS VULNERABILITY

A weak password policy means the application lets users set passwords like 123456 or password or admin. This sounds obvious but keeps appearing in production apps everywhere — especially internal tools, legacy systems, and startups. When an application accepts these passwords and has no account lockout, an attacker can write a script that tries the most common passwords across thousands of accounts. This is called credential stuffing or password spraying and it successfully breaks into a significant percentage of accounts because a large portion of real users choose exactly these weak passwords. For any application handling sensitive data, this is not just a policy issue — it is a direct path to mass account compromise.

HOW TO FIND IT — STEP BY STEP

Try to create a new account and set progressively weaker passwords: first try just the letter a, then 12, then 123, then 1234, then 12345, then 123456, then the word password, then admin, then your own username as the password. Document exactly which ones the application accepts. Then log out and go to the login page. Try common default credentials: admin and admin, admin and password, admin and 123456, test and test, user and user. In Burp Suite, capture a login request, send it to Intruder, set the password as the payload position, load the rockyou.txt or common-passwords.txt wordlist, and run the attack watching for any 200 response that differs from the normal failed login response.

TOOLS TO USE

Burp Suite Intruder | Hydra | SecLists wordlist

TC-09 No Brute Force Protection on Login

WHAT IS THIS VULNERABILITY

Brute force protection means the application limits how many wrong password attempts are allowed before taking defensive action — locking the account temporarily, requiring a CAPTCHA, adding a delay, or blocking the IP. Without any of these protections, an attacker can run an automated script that tests hundreds of thousands of password combinations. Modern hardware can test millions of combinations per second if the server does not slow them down. Even with a strong password policy, brute force will eventually succeed given enough time and no protective measures in place.

HOW TO FIND IT — STEP BY STEP

In Burp Suite, make one failed login attempt and send that request to Intruder. In the Intruder tab, highlight the password value and mark it as the injection point. Set payload type to Simple List and load a wordlist — SecLists has excellent ones. Set thread count to 50 and start the attack. Watch the response panel carefully. Count how many requests you send before hitting a CAPTCHA, a 429 Too Many Requests status, or an account lockout message. If you successfully send 100 or more requests and the server keeps returning the same failed-login response for each one without any lockout triggering, brute force protection is absent. Also test from a second IP address to check whether per-IP rate limiting exists separately from per-account lockout.

TOOLS TO USE

Burp Suite Intruder | Hydra | Medusa

TC-10 Session Token Exposed in URL

WHAT IS THIS VULNERABILITY

When you log into a website the server gives your browser a secret token that proves you are authenticated — normally stored as a cookie that the browser sends automatically. Some developers instead put this token directly in the URL — like https://example.com/dashboard?session=abc123xyz. This feels convenient but is a serious security problem. URLs are stored in browser history, readable by other users on a shared computer. They appear in the Referer header sent to external websites when you click any link. They show up in web server access logs. They get shared accidentally when someone copies and pastes a URL. Any one of these channels exposes the session token to an attacker who can then log in as the victim without ever knowing their password.

HOW TO FIND IT — STEP BY STEP

Open Burp Suite and browse the application normally after logging in. In the HTTP history, carefully examine every single request URL. Search for patterns like ?token= or ?session= or ?auth= or ?sid= in the URL path or query string. Also check every redirect response because tokens sometimes appear briefly in redirect URLs before the browser follows them. Check the browser address bar yourself while navigating through the app. Additionally view the page source and search for any tokens embedded as URL parameters in links within the HTML. If you find even one session token or JWT appearing as a URL parameter, the vulnerability is confirmed and you should document exactly which endpoints expose it.

TOOLS TO USE

Burp Suite | browser address bar | browser developer tools

TC-11 JWT None Algorithm Attack

WHAT IS THIS VULNERABILITY

JSON Web Tokens have a header that tells the server which cryptographic algorithm was used to create the signature — the signature is what prevents tampering. If you change the payload, the signature becomes invalid and the server should reject the token. However, JWT has a mode called none where no algorithm is used and no signature is required at all. This was meant for internal trusted systems but many implementations mistakenly accept it from untrusted clients. If a server accepts a JWT with algorithm set to none, an attacker can create any token they want — any user ID, any role, any claims — with zero signature and the server will trust it completely. This means instant access to any account including admin.

HOW TO FIND IT — STEP BY STEP

Capture any JWT from the application — look in the Authorization header, cookies, or request body. Copy the entire token and go to jwt.io. Paste it into the Encoded box on the left side. You will see the decoded header, payload, and signature on the right. Note the user ID and role in the payload. Now craft a modified token. Take the header and change the alg value from HS256 to none. Base64url-encode the modified header. Keep or modify the payload — for example change role from user to admin. Base64url-encode your desired payload. Concatenate everything as: modified-header dot modified-payload dot empty-string — that trailing dot is critical, it represents the empty signature. Send this crafted token in the Authorization header to a privileged endpoint. If the server accepts it and returns admin data, the none algorithm attack is confirmed.

TOOLS TO USE

jwt.io | Burp Suite | jwt_tool

TC-12 JWT Weak Secret Key

WHAT IS THIS VULNERABILITY

HS256 JWT tokens are signed using a secret key — a string only the server should know. If an attacker figures out this secret, they can create their own tokens with any payload they choose. They can make themselves an admin, impersonate any user, or set an expiry date 100 years in the future. The secret becomes the master key to the entire authentication system. Weak secrets are shockingly common — developers use placeholder values during development and forget to change them. Common culprits include: the word secret, the word password, the application name, the company name, a short number, or even an empty string. These crack in seconds with a dictionary attack.

HOW TO FIND IT — STEP BY STEP

Capture a valid JWT from the application and copy the complete token string. Open a terminal and run jwt_tool with the crack command: python3 jwt_tool.py YOUR_TOKEN_HERE -C -d /path/to/wordlist.txt using the jwt_secrets.txt wordlist from the jwt_tool repository. Alternatively run hashcat with mode 16500: hashcat -a 0 -m 16500 YOUR_TOKEN /usr/share/wordlists/rockyou.txt. If either tool finds the secret, it prints it to output. Go back to jwt.io, paste your token, enter the discovered secret in the signature verification box, and confirm it says Signature Verified. Now change the payload — set userId to 1 or the admin ID, change role to admin. Sign with the cracked secret. Send the forged token to admin endpoints and document the full account takeover with screenshots.

TOOLS TO USE

jwt_tool | hashcat mode 16500 | jwt.io

TC-13 Session Fixation

WHAT IS THIS VULNERABILITY

Session fixation is an attack where the attacker tricks the server into using a session ID that the attacker already knows. Here is how it works: the attacker visits the login page and gets a session ID assigned to them as an unauthenticated visitor. The attacker sends the victim a link to the application that includes this known session ID. The victim clicks the link, sees the normal login page, and enters their valid credentials. If the application does not generate a brand new session ID after the login succeeds — if it keeps using the same ID the attacker provided — then the attacker's browser and the victim's browser now share the same authenticated session. The attacker is logged in as the victim without ever knowing their password.

HOW TO FIND IT — STEP BY STEP

This test is simple and requires no special tools. Open the application in your browser and go to the login page without logging in. Open developer tools, go to the Application tab, then Cookies. Write down the exact current value of the session cookie. Now log in with valid credentials. After the login succeeds and you reach the dashboard, go back to developer tools and Cookies. Read the session cookie value again. If it is the exact same value as before you logged in, the application is vulnerable to session fixation. A secure application must always generate a completely new random session ID the instant a user successfully authenticates. Also test whether you can manually preset a session ID in the URL parameter and whether the server honors it.

TOOLS TO USE

Browser developer tools | Burp Suite | manual testing

TC-14 Insecure Password Reset Flow

WHAT IS THIS VULNERABILITY

The password reset feature is one of the most commonly attacked parts of a web application because developers often treat it as secondary and implement it quickly without thinking through all the security implications. Problems include: reset tokens that are too short and guessable, tokens that never expire so old links from months ago still work, tokens that are not invalidated after use so you can reset the password multiple times with the same link, and error messages that tell attackers whether an email address is registered in the system — leaking your entire user database through the forgot password page.

HOW TO FIND IT — STEP BY STEP

Request a password reset for an account you control and examine the email link carefully. Inspect the token portion — is it short, is it numeric only, does it look sequential or timestamp-based? Note exactly when you requested it. Wait 24 hours and try using the same link — if it still works, expiry is not enforced. Use the link once to reset the password, then try using the exact same link again to reset to yet another password — if it works, tokens are not invalidated after use. In Burp Suite, if the token is a short number like 6 digits, send it to Intruder with a numeric payload from 000000 to 999999 and attempt to brute force it. Also test whether the forgot password page returns different messages for registered versus unregistered emails — any difference confirms user enumeration.

TOOLS TO USE

Burp Suite | manual testing | email link inspection

Category: Sensitive Data

TC-15 Sensitive Data in Browser Local Storage

WHAT IS THIS VULNERABILITY

Browsers give web applications a local data store called localStorage. It persists after you close the browser and is accessible from any JavaScript running on the same domain. The problem is that any JavaScript on the page can read it — including a malicious script injected through an XSS vulnerability. Many developers store JWT tokens, session IDs, and user profile data in localStorage because it is easy to use with single-page applications. If there is even one XSS vulnerability anywhere on the same domain, an attacker can steal everything from localStorage with one line of JavaScript, send it to their server, and take over every active session.

HOW TO FIND IT — STEP BY STEP

Log into the application. Right-click anywhere and choose Inspect. Click the Application tab in DevTools. On the left sidebar expand Local Storage and click the application domain. You see every key-value pair stored there. Read every single one carefully. Look for: a JWT token which has three Base64 parts separated by dots, a long random string that might be a session ID, user personal details like email or user ID, API keys, or any security-sensitive configuration values. If you find any authentication token stored here, note that any XSS on this domain immediately results in session hijacking through a one-liner JavaScript payload. Also check Session Storage and IndexedDB in the same Application panel.

TOOLS TO USE

Browser DevTools Application tab | manual inspection

TC-16 Missing HTTPS or Weak TLS

WHAT IS THIS VULNERABILITY

HTTPS encrypts traffic between your browser and the server so that even if someone monitors your network connection — on public WiFi at a coffee shop, at an airport — they cannot read your passwords, tokens, or personal data. Without HTTPS everything travels as plain readable text. An attacker on the same network using Wireshark can see every single byte including your login credentials and session cookies. Even applications that use HTTPS can have weaknesses — using outdated TLS versions like TLS 1.0 or 1.1 with known vulnerabilities, or accepting weak cipher suites.

HOW TO FIND IT — STEP BY STEP

The first check is simply looking at the URL bar. If the login page or any page handling sensitive data starts with http:// instead of https://, the finding is immediately confirmed. Then go to ssllabs.com/ssltest/ and run a full SSL analysis against the target domain — it gives a grade from A+ to F and shows exactly which vulnerabilities exist including weak ciphers, outdated TLS versions, and certificate problems. In Burp Suite, check whether any HTTP links exist inside the HTTPS application. Also test manually: type the target URL with http:// and see if it redirects to https:// or serves content over plain HTTP. Use Wireshark on a controlled test network to capture actual traffic and confirm whether credentials are visible in packet captures.

TOOLS TO USE

Wireshark | Burp Suite | SSL Labs scanner

TC-17 Sensitive Data in Error Messages

WHAT IS THIS VULNERABILITY

When an application encounters an unexpected error — a failing database query, a file it cannot find, an input it cannot process — it needs to handle that gracefully. The problem is when developers leave debug mode enabled in production or forget to implement proper error handling. Instead of showing a friendly error page, the application dumps the entire technical error to the screen including the database query that failed, the full server file path, the database server name and version, the application framework and version, internal IP addresses, or a complete stack trace showing every line of code that executed. Each piece of this information helps an attacker craft more targeted attacks against the specific versions and configurations they can now see.

HOW TO FIND IT — STEP BY STEP

Deliberately try to break the application in every way you can think of. Change numeric IDs in URLs to letters: ?id=abc instead of ?id=123. Change ?id=5 to ?id=99999999. Submit forms with completely empty required fields. Paste a 5000 character string into a phone number field. Enter special characters like single quotes, angle brackets, null bytes, and percent signs into every field. Use Burp Suite to change Content-Type headers to unexpected values and change numeric JSON values to strings and vice versa. After each of these actions, read the full response page extremely carefully. Look specifically for: technology names and version numbers, server path strings like /var/www/html, database table or column names, any stack trace showing code file names and line numbers, or internal IP addresses.

TOOLS TO USE

Burp Suite | OWASP ZAP | manual input fuzzing

TC-18 Plaintext Password Storage

WHAT IS THIS VULNERABILITY

Passwords should be stored in a way that even if the entire database is stolen, the attacker cannot figure out the actual passwords. The correct approach is hashing with a strong algorithm like bcrypt, Argon2, or scrypt — one-way functions that cannot be reversed and include a salt to prevent rainbow table attacks. Plaintext storage means passwords are saved exactly as typed — anyone who reads the database reads every password immediately. Base64 encoding is equally bad because it is not encryption at all, it is just a different way of writing the same data and decodes in one second. MD5 without salting is almost as bad — modern graphics cards can test 10 billion MD5 hashes per second and most common passwords crack in milliseconds against rainbow tables.

HOW TO FIND IT — STEP BY STEP

This finding is typically confirmed through database access obtained via SQL injection or a disclosed backup file. Use SQLmap to extract the users table: sqlmap -u TARGET_URL -p PARAM --dump -T users. Look at the password column values. If they look like readable English words or email addresses, they are stored in plaintext — immediately critical. If they are exactly 32 characters of hexadecimal, they are likely MD5 — paste one into crackstation.net and if it cracks instantly, the finding is critical. If passwords are 60 character strings starting with $2y$, that is bcrypt and is properly secured. Document exactly which hashing algorithm is used and whether salting is present by examining multiple hashes for the same simple password.

TOOLS TO USE

SQLmap | CrackStation | hashid | hashcat

TC-19 Weak Cryptographic Algorithm

WHAT IS THIS VULNERABILITY

Not all cryptographic algorithms provide equal protection. MD5 was once used for password hashing but is now completely broken for security purposes — it is so fast to compute that modern graphics cards test 10 billion MD5 hashes per second. SHA1 has known theoretical weaknesses and is no longer recommended for sensitive applications. DES and 3DES are symmetric encryption algorithms considered too weak due to short key lengths. Using these old algorithms provides a false sense of security — data appears protected but an attacker with moderate resources can break the encryption in hours or days.

HOW TO FIND IT — STEP BY STEP

Look for hash values appearing anywhere in the application — in database dumps obtained through SQL injection, in API responses, in cookies, or in password reset tokens. Copy a sample hash and paste it into hashid or visit hashes.com to identify the algorithm automatically. MD5 hashes are always exactly 32 hexadecimal characters. SHA1 hashes are exactly 40 characters. If you identify MD5 or SHA1 hashes used for passwords, paste several into crackstation.net which has massive precomputed rainbow tables — if common passwords crack instantly, the finding is confirmed critical. For SSL and TLS configuration, use SSL Labs to check which cipher suites the server accepts. Any report of DES, RC4, or export-grade ciphers is a confirmed weakness.

TOOLS TO USE

hashid | hashcat | CrackStation | SSL Labs | nmap

TC-20 Full Credit Card Number Exposure

WHAT IS THIS VULNERABILITY

Payment card data is among the most strictly regulated personal data in the world under PCI-DSS standards. A fundamental rule is that primary account numbers — the 16-digit card numbers — must never be stored, transmitted, or displayed in full after the initial transaction is authorized. Applications should store only the last 4 digits for display and use tokenization for recurring payments. When an application stores or returns full card numbers in API responses, database records, or on-screen receipts, it creates enormous liability — a single database breach exposes every customer's financial data directly, enabling immediate fraud without any additional cracking by the attacker.

HOW TO FIND IT — STEP BY STEP

Complete a test transaction using a test card number if the application has a sandbox mode, or the lowest possible real transaction with permission. After completion, methodically check every single API response in Burp Suite history that was generated during and after the checkout flow. Search specifically for patterns of 13 to 16 consecutive digits in all response bodies — these match credit card number formats. Also check the transaction history page, order confirmation page, invoice download PDF, and any receipt email triggered by the purchase. In the database if accessible via SQL injection, query the payments or transactions table. Any full 16-digit card number appearing anywhere outside the initial payment submission confirms this critical PCI-DSS violation.

TOOLS TO USE

Burp Suite | manual payment flow inspection

Category: XXE

TC-21 XML External Entity Injection (XXE)

WHAT IS THIS VULNERABILITY

XML has a feature called external entities — a way to define a shorthand symbol that the XML parser should replace with content from somewhere else. That somewhere else can be a local file on the server, a network resource, or an internal service URL. When a web application accepts XML input from a user and the underlying XML parser has external entity processing enabled — which is unfortunately the default in many older parsers — an attacker can inject an entity definition that tells the parser to read any file on the server and embed its contents into the XML response. This means reading /etc/passwd to get server usernames, reading application configuration files containing database passwords, or reading cloud instance metadata to steal cloud credentials.

HOW TO FIND IT — STEP BY STEP

Find any functionality that accepts XML data — SOAP web services, REST API endpoints accepting application/xml content type, document upload features that handle XML or DOCX or SVG files. In Burp Suite, capture a request that sends XML. Inject an entity declaration at the top of the XML document. The payload structure is: a DOCTYPE declaration defining an entity that loads file:///etc/passwd, then a reference to that entity inside an XML element in the body. Send the request and inspect the response body. If you see the contents of /etc/passwd with entries like root:x:0:0 appearing in the response, XXE is confirmed. Escalate by reading application config files like /var/www/html/config.php or /etc/environment which often contain database passwords and API keys.

TOOLS TO USE

Burp Suite | OWASP ZAP | XXEinjector

TC-22 Blind XXE via Out-of-Band Channel

WHAT IS THIS VULNERABILITY

Blind XXE is the same vulnerability as regular XXE but the server processes the malicious XML and reads the requested file without including the contents in the HTTP response. The application might just return a success message. To confirm and exploit blind XXE, attackers use out-of-band communication. Instead of reading a file and returning it in the response, the payload makes the server send the file contents to an external server the attacker controls — using an HTTP request or DNS lookup. The attacker owns that receiving server so they see the data arrive there even though nothing appeared in the application's response.

HOW TO FIND IT — STEP BY STEP

Set up an out-of-band listener before testing. In Burp Suite Professional, open Collaborator Client to get your unique subdomain. Without Burp Pro, run interactsh-client in your terminal for a free unique URL. Copy your unique subdomain. Now find any endpoint accepting XML and inject a payload using a SYSTEM identifier pointing to your collaborator URL instead of a local file path. Send the request. Wait 30 to 60 seconds then check your Collaborator or interactsh dashboard. If you see any incoming DNS lookup or HTTP request originating from the target server's IP address, blind XXE is confirmed — the server attempted an outbound connection based entirely on your injected XML entity definition.

TOOLS TO USE

Burp Suite Collaborator | interactsh | manual testing

Category: Broken Access Control

TC-23 Insecure Direct Object Reference (IDOR)

WHAT IS THIS VULNERABILITY

IDOR is one of the most common and impactful vulnerabilities in bug bounty programs. Every resource in a web application has an identifier — your user account might be user ID 4218, your order might be order ID 88921. When an application lets you access resources by including this ID in the URL or request but does not check whether the logged-in user actually owns that specific ID, an attacker simply changes the number and accesses someone else's data. Change your user ID to 4217 and you see the previous user's account. Change the order ID and you see another person's purchase history. The application trusts the ID completely without ever asking if you own that resource.

HOW TO FIND IT — STEP BY STEP

Log into the application and browse through every feature that retrieves user-specific data. In Burp Suite watch the HTTP history and identify every request containing a numeric ID, UUID, username, or email address in the URL or request body. Make a list of these endpoints. For each one, change the identifier to a different value — increment a number, try nearby values, substitute another user's email. Focus on: profile view endpoints, order detail endpoints, invoice download endpoints, API calls returning account information, and any export or download feature. If another user's data comes back in the response, IDOR is confirmed. The most impactful IDOR findings involve financial data, health records, or private messages.

TOOLS TO USE

Burp Suite | manual testing | Autorize Burp extension

TC-24 Horizontal Privilege Escalation

WHAT IS THIS VULNERABILITY

Horizontal privilege escalation is where you stay at the same privilege level — regular user to regular user — but gain access to a different user's resources without their permission. This is different from vertical escalation where you go from user to admin. Horizontal escalation is about moving sideways across accounts. The root cause is almost always the same as IDOR — the application looks up data based on an identifier from the request and serves it to whoever asks, without verifying the requesting user has any legitimate relationship to that data. This is particularly dangerous in financial applications where one account can view or modify another account's transactions.

HOW TO FIND IT — STEP BY STEP

You need two separate test accounts. Log in as User A in one browser and User B in another. In Burp Suite with User A's session active, capture a request that accesses User A's personal data. Note User A's session cookie value. Now take that same request in Burp Repeater and replace User A's session cookie with User B's session cookie value. Send the modified request. If you receive User A's data back while using User B's session, the server is not checking whether the authenticated user matches the resource being requested. Document exactly which endpoints are affected and what sensitive data categories are exposed.

TOOLS TO USE

Burp Suite | Autorize extension | two test accounts

TC-25 Vertical Privilege Escalation to Admin

WHAT IS THIS VULNERABILITY

Vertical privilege escalation is a regular user gaining administrator access. This can happen several ways: admin-only URL paths that are hidden from the UI but have no actual server-side authorization check. Applications that trust a role parameter from the client — an attacker changes role=user to role=admin in a cookie, POST parameter, or JWT payload. Or applications that check permissions on main pages but forget to check them on the underlying API endpoints those pages call. Vertical escalation typically means the most severe possible impact — full control of all user data, ability to delete accounts, access to system configuration, and sometimes code execution through admin features.

HOW TO FIND IT — STEP BY STEP

Start with direct URL navigation to common admin paths while logged in as a regular user: /admin, /admin/dashboard, /admin/users, /administrator, /panel, /cp, /management, /api/admin, /api/v1/admin/users. Document which paths return content instead of a 403 error. In Burp Suite, look for any request parameter containing: role, privilege, isAdmin, userType, or access. Try changing these to admin, true, 1, or ADMIN and resend. If using JWT tokens, decode on jwt.io, check for any role claim, and if you can crack the signing secret, generate a new token with elevated role. Use the Autorize Burp extension — it automatically replays every request you make as an admin using a lower-privilege token to detect authorization gaps systematically.

TOOLS TO USE

Burp Suite | dirsearch | Autorize extension | jwt_tool

TC-26 Missing Function Level Access Control

WHAT IS THIS VULNERABILITY

Function level access control means every individual endpoint and feature checks whether the requesting user has permission to use it. In large applications with many developers, it is easy for some endpoints to be deployed to production without proper authorization checks. This is especially common with administrative API endpoints not linked from any user interface — the developer assumes that since no regular user knows the URL, no one will find it. But attackers discover hidden endpoints through JavaScript file analysis, directory brute forcing, reviewing API documentation, or simply guessing logical endpoint names based on what they can see in the application.

HOW TO FIND IT — STEP BY STEP

Use directory fuzzing to enumerate hidden paths. Run dirsearch: dirsearch -u https://target.com -e php,asp,json -w /usr/share/wordlists/dirb/big.txt. Focus on paths containing: admin, manage, internal, private, console, api, config, debug, test, backup, export, or report. When you find interesting paths, access them while logged in as a regular user. Also open every JavaScript file the application loads in your browser and use the Search function to find all URL strings and API endpoint patterns — these frequently reveal undocumented endpoints. Use Burp Suite Engagement Tools to extract all links from the application sitemap automatically.

TOOLS TO USE

dirsearch | ffuf | gobuster | Burp Suite

TC-27 CORS Misconfiguration

WHAT IS THIS VULNERABILITY

CORS controls which websites can make JavaScript requests to your API. By default browsers block a script on evil.com from making requests to api.bank.com. CORS lets servers explicitly whitelist allowed origins. The problem comes when developers configure CORS too permissively — reflecting back whatever Origin header the request sends, or accidentally allowing null origins. A misconfigured CORS policy on an authenticated API means an attacker can create a malicious webpage that uses a victim's browser to make authenticated API calls to the target, collect the responses containing the victim's sensitive data, and send it all to the attacker — while the victim simply views what looks like a harmless page.

HOW TO FIND IT — STEP BY STEP

In Burp Suite, capture any authenticated API request and open it in Repeater. Add this custom header: Origin: https://attacker.com. Send it and look at the response headers. If you see Access-Control-Allow-Origin: https://attacker.com in the response, CORS is reflecting arbitrary origins. Check also whether Access-Control-Allow-Credentials: true appears — this combination is the most dangerous because it means the browser will include the victim's cookies in cross-origin requests, enabling full authenticated data theft. Also test Origin: null as an alternative bypass. Write a proof-of-concept HTML page using the Fetch API that makes a cross-origin request to a sensitive endpoint and logs the response to the browser console to demonstrate real-world exploitability.

TOOLS TO USE

Burp Suite | CORSy | manual Origin header testing

Category: Security Misconfiguration

TC-28 Default Credentials Left in Place

WHAT IS THIS VULNERABILITY

Every piece of software ships with default factory credentials for initial setup. The assumption is administrators change these immediately. In reality this often does not happen — especially on internal tools, development servers promoted to production, or software installed during emergencies. Default credentials databases like CIRT.net catalog the defaults for thousands of products. Attackers who identify what software is running can look up the defaults instantly. This is particularly common with database administration panels like phpMyAdmin, monitoring tools like Grafana, network devices, IoT devices, and content management systems.

HOW TO FIND IT — STEP BY STEP

First identify what software is running using Wappalyzer browser extension — it auto-detects technologies and versions as you browse. Check HTTP response headers for server and framework information. Once you know the software, search CIRT.net or Google for: software-name default credentials. Common ones to always test regardless of software: admin/admin, admin/password, admin/1234, root/root, root/toor, administrator/administrator, test/test, guest/guest, and blank password with admin username. Also check if a setup wizard or first-run page is still accessible at /install or /setup. In Burp Suite, use Intruder with a dedicated default-credentials wordlist from the SecLists repository to automate testing many combinations quickly.

TOOLS TO USE

CIRT.net | Hydra | Burp Suite Intruder | Wappalyzer

TC-29 Directory Listing Enabled

WHAT IS THIS VULNERABILITY

When a web server gets a request for a directory URL with no index file present, it can either return a 403 Forbidden response (secure) or display an auto-generated HTML page listing every file and folder in that directory (insecure). This exposure frequently leads to finding backup files, archived source code copies, exported database dumps, uploaded user files, configuration files, and log files that were never meant to be public. Attackers specifically look for this because one directory listing can reveal the entire structure of the application and provide access to files that should be completely inaccessible.

HOW TO FIND IT — STEP BY STEP

Browse directly to the most likely paths that have files but no index page: /uploads/, /backup/, /backups/, /files/, /assets/, /images/, /docs/, /logs/, /tmp/, /temp/, /exports/, /downloads/. Try each one in your browser. If you see an Apache or Nginx auto-generated listing page showing file names, sizes, and modification dates, directory listing is confirmed for that path. Then look through every listed file — download anything that looks interesting, specifically .zip, .tar.gz, .sql, .bak, .log, and .conf files. Use dirsearch to enumerate more paths automatically. Document all sensitive files accessible and calculate the full impact of what an attacker could learn from the exposed directory.

TOOLS TO USE

Browser navigation | dirsearch | Nikto

TC-30 Sensitive and Backup Files Publicly Accessible

WHAT IS THIS VULNERABILITY

Developers leave surprisingly many sensitive files in web-accessible directories. The .env file is extremely common in modern applications and contains database credentials, API keys, JWT secrets, and cloud access keys — essentially every credential the application needs to run. The .git directory contains the entire source code history — if accessible, an attacker downloads the complete codebase and reads every credential ever committed, even ones later deleted. Backup files contain complete snapshots of the production database. These files are so commonly exposed that professional penetration testers check for them within the first five minutes of every single engagement.

HOW TO FIND IT — STEP BY STEP

Manually check each of these paths in your browser: /.env, /.env.backup, /.env.local, /.env.production, /.git/config, /.git/HEAD, /config.php, /configuration.php, /wp-config.php, /database.yml, /settings.py, /appsettings.json, /web.config, /.htaccess, /backup.zip, /backup.sql, /dump.sql, /db.sql. For any file that returns a 200 response, download it immediately and read the contents thoroughly. If /.git/HEAD is accessible, use GitDumper to reconstruct the entire repository: git-dumper https://target.com/.git ./output-folder. This downloads every source file. Then run truffleHog against the downloaded repository to automatically find all secrets, API keys, and credentials ever committed to the git history.

TOOLS TO USE

Browser manual testing | GitDumper | truffleHog | dirsearch

TC-31 HTTP Security Headers Missing

WHAT IS THIS VULNERABILITY

HTTP security headers are instructions a server sends with every response telling the browser how to behave securely. Content-Security-Policy prevents XSS by controlling which scripts are allowed to run.
X-Frame-Options prevents clickjacking by forbidding the page from loading inside an iframe.
Strict-Transport-Security tells the browser to always use HTTPS for this domain and never downgrade.
X-Content-Type-Options prevents the browser from guessing at file types. When these headers are absent, the browser uses its default permissive behavior and all the listed attack classes become exploitable.

HOW TO FIND IT — STEP BY STEP

Open Burp Suite and browse the application while it captures traffic. In the HTTP history, click any response from the target domain and look at the Headers section. Go through this checklist of headers that should be present: Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy. Note every missing one. For a faster comprehensive check, go to securityheaders.com and enter the target URL — it gives an A through F grade and explains every missing header and which attack each one prevents. Also check Set-Cookie headers for session cookies — they must include both the Secure flag and the HttpOnly flag. Missing Secure means cookies travel over HTTP; missing HttpOnly means JavaScript can read the cookie enabling XSS-based session theft.

TOOLS TO USE

Burp Suite | securityheaders.com | OWASP ZAP

TC-32 Open Redirect Vulnerability

WHAT IS THIS VULNERABILITY

An open redirect happens when an application accepts a URL as a parameter and redirects the user to it without any validation. This might exist legitimately — after login, redirect the user to the page they were trying to access via ?next=/dashboard. The problem is when the application redirects to any URL including external ones. Attackers use open redirects for phishing: they craft a link starting with the legitimate trusted domain but ending with a redirect to an attacker-controlled site that looks identical to the real login page. Because users check the domain name for legitimacy, seeing their company's real domain in the URL makes them trust the link completely.

HOW TO FIND IT — STEP BY STEP

Search through the entire application for URL parameters that look like redirect destinations. Common parameter names: redirect, redirect_url, next, return, return_url, url, goto, target, destination, redir, callback, and continue. Find these in URL query strings or POST request bodies. For each one, replace the current value with https://google.com. Follow the request in your browser and see where you end up. If you land on google.com after hitting the application endpoint, open redirect is confirmed. Then test with your own domain or a Burp Collaborator URL to show the redirect works to any arbitrary external destination. Document the exact crafted URL that triggers the redirect for your proof of concept.

TOOLS TO USE

Burp Suite | manual browser testing | OWASP ZAP

TC-33 Clickjacking Vulnerability

WHAT IS THIS VULNERABILITY

Clickjacking is an attack where the attacker creates a webpage that embeds the target website inside a transparent or invisible iframe positioned precisely over some decoy content. The victim thinks they are clicking on something innocent but their clicks land on buttons and links on the invisible target page underneath. Attacks can make users unknowingly confirm transactions, change account settings, make purchases, or authorize actions they have no idea they are performing. The defense is the server sending X-Frame-Options: DENY or Content-Security-Policy with frame-ancestors none.

HOW TO FIND IT — STEP BY STEP

Create a simple HTML file locally with an iframe pointing to the target URL, styled with CSS opacity of 0.001 and positioned as an overlay on some dummy content. Open this file in your browser. If the target website loads and is faintly visible inside your page, clickjacking is possible. If the browser shows an empty frame or a refused message, the site is properly protected. Faster check: in Burp Suite look at any response from the target and check whether X-Frame-Options header is present. If it is missing and Content-Security-Policy also lacks a frame-ancestors directive, the protection is absent. Use clickjacker.io for an automated one-click test against any URL.

TOOLS TO USE

Browser manual HTML test | Burp Suite | clickjacker.io

Category: XSS

TC-34 Reflected Cross-Site Scripting

WHAT IS THIS VULNERABILITY

Reflected XSS is an injection attack where a malicious JavaScript payload is embedded in a URL parameter, sent to the server, and the server includes that payload in the HTML response without sanitizing it. The victim's browser encounters the script tag and executes the JavaScript. The attacker needs to trick the victim into clicking a specially crafted link that contains the payload in the URL — hence reflected. The payload can steal the session cookie and send it to the attacker's server, enabling complete account takeover without the victim noticing anything beyond a brief normal page load.

HOW TO FIND IT — STEP BY STEP

Test every URL parameter in the application. For a URL like /search?q=hello, change the q value to a simple test payload starting with just a less-than sign followed by the word test. If you see <test as literal HTML-encoded text in the page source, the character is being safely encoded. If you see a less-than sign actually rendered, escalation is possible. Try a full script tag with alert(1) inside. If the browser shows an alert dialog, reflected XSS is confirmed. If script tags are filtered, try: SVG onload handlers, image onerror handlers, or JavaScript protocol in href attributes. Use XSStrike and Dalfox which test hundreds of payloads and bypass techniques specifically designed to evade common filters and WAFs.

TOOLS TO USE

Burp Suite | OWASP ZAP | XSStrike | Dalfox

TC-35 Stored Cross-Site Scripting

WHAT IS THIS VULNERABILITY

Stored XSS is the most dangerous form of XSS because the malicious script is permanently saved in the database and executes for every user who views the affected page — not just the person who submits the payload. The attacker does not need to trick each victim individually. They submit the payload once into a comment section, user profile, product review, or support ticket. From that point forward, every user who views that page executes the attacker's script. The most impactful stored XSS payloads steal session cookies and send them to an external server, creating an account takeover that spreads to every user who visits the infected page.

HOW TO FIND IT — STEP BY STEP

Identify every input field that saves data and displays it back on a page later — especially fields visible to other users. This includes: comment and review fields, profile names and bio fields, forum posts, message boards, support tickets, chat applications, custom tags, and admin-visible feedback forms. Submit a safe test string first: just type XSSTEST as the value and check where it appears. Then inject a script tag with an alert, or an image tag with an onerror event, or an SVG with an onload event. If you see an alert dialog execute when the page loads, stored XSS is confirmed. Immediately document it with a screenshot before clearing it, and note which user roles are affected. If admin users also see the alert, the impact is especially critical.

TOOLS TO USE

Burp Suite | XSStrike | OWASP ZAP | manual testing

TC-36 DOM-Based Cross-Site Scripting

WHAT IS THIS VULNERABILITY

DOM-based XSS exists entirely within browser-side JavaScript. The server is not involved in the injection at all. The vulnerability happens when a JavaScript function reads data from an attacker-controllable source — like the URL fragment after the hash symbol, the document referrer, or window.name — and then writes that data directly into the page DOM using a dangerous function like innerHTML, document.write, or eval without sanitizing it. Because the payload never reaches the server, server-side input validation cannot stop it. The payload typically lives in the URL after the hash character since hash fragments are never sent to the server.

HOW TO FIND IT — STEP BY STEP

Analyzing DOM-based XSS requires examining the application's JavaScript source code. Open developer tools, go to Sources, and load each JavaScript file. Use the Search function across all files and search for dangerous sink functions: innerHTML, outerHTML, document.write, eval, setTimeout with a string argument, and location assignments. When you find one, trace backwards to see what value is being written to it. If that value comes from location.hash, location.search, location.href, document.URL, document.referrer, or any cookie value, you have a potential source-to-sink connection. Use Burp Suite's DOM Invader browser extension to automate this analysis — it instruments the DOM sources and automatically detects when attacker-controlled data reaches a dangerous sink.

TOOLS TO USE

Browser DevTools | Burp DOM Invader | manual JS review

Category: Insecure Deserialization

TC-37 Insecure Deserialization Leading to RCE

WHAT IS THIS VULNERABILITY

Serialization converts an in-memory object — a user session, a shopping cart, a permission set — into a format that can be stored or transmitted. Deserialization is the reverse. The vulnerability occurs when an application deserializes data from an untrusted source like a cookie value or POST parameter without verifying it has not been tampered with. In Java, PHP, Python, Ruby, and .NET, the deserialization process can automatically trigger the execution of methods in certain classes. Attackers craft malicious serialized payloads using known vulnerable class chains — called gadget chains — that cause the deserialization process itself to execute arbitrary operating system commands.

HOW TO FIND IT — STEP BY STEP

Look for Base64-encoded data in cookies, POST body parameters, or HTTP headers that looks more complex than a simple session token. Java serialized objects always start with the bytes AC ED 00 05, which Base64-encodes to rO0AB at the beginning — if you see a cookie or parameter starting with rO0AB, it is almost certainly a Java serialized object. For PHP, serialized data starts with O: followed by a number for objects or a: for arrays. Once you identify serialized data, use ysoserial for Java targets to generate payloads for known gadget chains — start with CommonsCollections1 through CommonsCollections7. Encode the payload and substitute it for the original serialized value. If the server executes a command like a DNS lookup to your Burp Collaborator, deserialization RCE is confirmed.

TOOLS TO USE

ysoserial | Burp Suite | manual byte inspection

TC-38 PHP Object Injection

WHAT IS THIS VULNERABILITY

PHP has a built-in function called unserialize() that reconstructs PHP objects from a string. If an application passes user-controlled data to unserialize() — from a cookie, POST parameter, or database value — an attacker can craft a malicious PHP object that when reconstructed triggers a dangerous magic method. PHP magic methods like __destruct, __wakeup, and __toString are automatically invoked by the PHP runtime at specific points — when an object is destroyed, when it is created, or when it is converted to a string. By crafting an object of a class that exists in the application codebase where one of these methods performs a dangerous action, the attacker achieves code execution purely through the deserialization lifecycle.

HOW TO FIND IT — STEP BY STEP

Look through all cookies and POST parameters for any data matching PHP serialization syntax. The telltale pattern is strings starting with O: followed by a number and a colon and a quoted class name — for example O:4:User:2:{...} — or a: for arrays. If you find any, copy the value and manually modify parts of the object — change a string value, alter a numeric property, or change the class name. Resend the modified request and observe the response for any change in behavior, error messages, or unexpected output. Use the PHP Object Injection Scanner Burp extension to automate detection. If the application is open source, download the codebase and search for all calls to unserialize() — then map which classes are available in that scope to identify exploitable gadget chains.

TOOLS TO USE

Burp Suite | PHP Object Injection Scanner extension | source code review

Category: Vulnerable Components

TC-39 Outdated Libraries with Known CVEs

WHAT IS THIS VULNERABILITY

Every web application is built on top of dozens or hundreds of third-party libraries and frameworks. When vulnerabilities are discovered in these components, maintainers release patches. But applications that are not actively maintained continue running vulnerable versions for months or years after patches are available. Public vulnerability databases like the National Vulnerability Database and exploit repositories like ExploitDB catalog these vulnerabilities with CVE numbers and often with public proof-of-concept exploits. Attackers look for version numbers exposed by applications because a known vulnerable version combined with a public exploit turns a complex attack into a simple copy-paste operation.

HOW TO FIND IT — STEP BY STEP

The first step is fingerprinting — identifying exactly which technologies and versions the application uses. Install Wappalyzer browser extension and browse the application — it automatically detects frameworks, CMS platforms, JavaScript libraries, and server software with version numbers. In Burp Suite, examine response headers for Server, X-Powered-By, and X-Generator headers which often contain version information. View page source and search for version strings in script src attributes and HTML comments. Once you have a list of technologies with versions, search each one in the format: TECHNOLOGY VERSION CVE on Google and also check nvd.nist.gov. Use Retire.js as a browser extension for automatic detection of outdated JavaScript libraries.

TOOLS TO USE

Wappalyzer | Retire.js | Nikto | NVD CVE database

TC-40 Outdated Web Server Software

WHAT IS THIS VULNERABILITY

Web servers and application servers announce their version numbers in HTTP response headers by default. Apache announces itself with something like Apache/2.4.29 in the Server header. Nginx does similarly. Tomcat, WebLogic, and JBoss all have default behaviors that expose their version numbers. Attackers who see Apache/2.4.29 immediately know exactly which CVEs apply — this version has multiple known vulnerabilities including a path traversal allowing file reading outside the web root. The combination of automatic version announcement and publicly available exploits means an unpatched server is essentially broadcasting a direct invitation.

HOW TO FIND IT — STEP BY STEP

Use curl to check the Server header: run curl -I https://target.com and read the Server header in the output. Note the exact version string. Check X-Powered-By and any other technology-revealing headers. Then search that exact version string in the National Vulnerability Database at nvd.nist.gov and in ExploitDB at exploit-db.com. Look specifically for vulnerabilities rated 7.0 or higher in CVSS score, and for those with public proof-of-concept exploit code. Run Nikto for a quick automated scan that checks version numbers against its built-in vulnerability database and reports findings with CVE references. Use Nmap with the -sV flag to enumerate server versions comprehensively across all open ports.

TOOLS TO USE

curl | Nmap with -sV | Nikto | ExploitDB | NVD

TC-41 Vulnerable WordPress Plugins and Themes

WHAT IS THIS VULNERABILITY

WordPress powers over 40 percent of all websites and its plugin ecosystem has over 60,000 plugins. Security quality varies enormously — many are maintained by individual developers who may not be security experts or may abandon the plugin entirely. When a vulnerability is published for a WordPress plugin — and WPScan catalogs tens of thousands of plugin vulnerabilities — every site running that version becomes a target. Common vulnerability types include SQL injection in shortcode parameters, XSS in comment fields, arbitrary file upload in media handlers, authentication bypass in custom login flows, and unauthenticated access to admin AJAX endpoints.

HOW TO FIND IT — STEP BY STEP

Run WPScan against the target WordPress site with aggressive detection: wpscan --url https://target.com --plugins-detection aggressive --api-token YOUR_TOKEN. The API token is free from WPScan and provides vulnerability data from their database. WPScan enumerates all installed plugins, detects their version numbers, and automatically flags known vulnerabilities with CVE references and links to proof-of-concept exploits. Also browse to /wp-content/plugins/ and check if directory listing is enabled — it often reveals plugin names and versions. For each vulnerable plugin found, look up the specific CVE on WPVulnDB and understand exactly what the vulnerability allows an unauthenticated attacker to do.

TOOLS TO USE

WPScan with API token | WPVulnDB | Burp Suite

Category: Logging & Monitoring

TC-42 Insufficient Security Logging and Monitoring

WHAT IS THIS VULNERABILITY

Security logging is recording events that matter from a security perspective — failed login attempts, access denied errors, account lockouts, privilege changes, unusual data access patterns, and detected attack attempts. Without this logging, an attacker can break into a system and operate within it for weeks or months without anyone knowing. When an incident is eventually discovered, the absence of logs means there is no way to understand what the attacker accessed, what they changed, how they got in, or how long they were active. This makes recovery much harder and regulatory consequences much more severe.

HOW TO FIND IT — STEP BY STEP

Test logging behavior by performing a series of clearly suspicious actions that any security system should detect and record. Attempt to log in with the wrong password 10 times for the same account. Try to access admin URLs while logged in as a regular user. Inject SQL characters into input fields. Access resources with IDs belonging to other users. Navigate to paths that do not exist. Then check application logs directly if you have access — or ask the security team. A properly configured system should have log entries for every failed login with timestamp, IP address, and username. It should log every 403 Forbidden response for attempts to access protected resources. If these events are absent, the application has no visibility into attacks happening against it in real time.

TOOLS TO USE

Manual testing | log file review | Splunk | ELK Stack

TC-43 Log Injection and Log Forgery

WHAT IS THIS VULNERABILITY

Log injection is a subtle attack where an attacker includes newline characters in input data that ends up written to log files. Log files are typically one line per event. If an attacker can inject a newline into a username field, they can start a new log line that appears to be a completely legitimate log entry — a fake successful admin login from a trustworthy-looking IP address. This hides their actual attack in a sea of forged legitimate-looking activity, making forensic investigation much harder. It can also confuse log analysis tools and security analysts who trust what they see in logs.

HOW TO FIND IT — STEP BY STEP

In Burp Suite, intercept any request where the submitted data is likely to appear in server logs — the username field on login, the User-Agent header, the Referer header, the X-Forwarded-For header, and any search or message field. In the username field inject: testuser followed by a URL-encoded newline character %0a followed by a completely fabricated log entry like: 2024-01-01 00:00:00 INFO Successful admin login from 192.168.1.1. Submit the request. Then check the application log files if you have access through another vulnerability like path traversal or an exposed log directory — look for whether your injected text created a fake additional log line appearing as a legitimate entry.

TOOLS TO USE

Burp Suite | manual request manipulation

Category: SSRF

TC-44 Server-Side Request Forgery (SSRF)

WHAT IS THIS VULNERABILITY

SSRF is a vulnerability where an attacker tricks the server into making HTTP requests to an arbitrary destination on their behalf. Web servers typically have much broader internal network access than end users — they can reach internal databases, caches, admin dashboards, and internal APIs that are completely inaccessible from the internet. When a server-side feature accepts a URL and fetches it — a webhook configuration, an image import by URL, a URL preview generator — an attacker can substitute an internal IP address. The server fetches the content and returns it. In cloud environments, the most critical SSRF target is the instance metadata service at 169.254.169.254 which returns cloud credentials that can be used to escalate access to the entire cloud infrastructure.

HOW TO FIND IT — STEP BY STEP

Search methodically through the entire application for any feature accepting a URL as input. Look for: webhook URL configuration in settings pages, profile image import from URL, document or link preview functionality, payment gateway callback URLs, email notification test features, export to external URL options, and any API parameter named url, fetch, image, webhook, callback, proxy, or destination. When you find one, start with: replace the URL with http://127.0.0.1 or http://localhost and observe the response. If you get a different response than when using an invalid URL — even just a different timeout versus an immediate error — the server is making outbound requests. Escalate immediately to http://169.254.169.254/latest/meta-data/ for AWS targets. Any cloud credentials returned represent a critical finding with potential for complete cloud infrastructure compromise.

TOOLS TO USE

Burp Suite Collaborator | SSRFmap | interactsh

TC-45 Blind SSRF via Out-of-Band Callbacks

WHAT IS THIS VULNERABILITY

Blind SSRF is the same fundamental vulnerability as regular SSRF — the server makes requests to attacker-specified destinations — but the content of those requests is not returned in the HTTP response. The application just shows a success message or nothing at all. To detect it, attackers use out-of-band interaction monitoring. They set up a server that logs all incoming connections including DNS lookups. They inject a URL pointing to their monitoring server as the SSRF target. If the application is vulnerable, the server attempts to resolve the DNS name and make an HTTP connection to the monitoring server. Those connections get logged, proving the SSRF exists even though nothing appeared in the application response.

HOW TO FIND IT — STEP BY STEP

Set up your out-of-band listener first. In Burp Suite Professional, click on Burp menu then Collaborator Client to get your unique collaborator subdomain. Without Burp Pro, run interactsh-client in your terminal for a free unique subdomain. Copy your unique URL. Now find all URL-accepting parameters as described in the regular SSRF test. For each one, replace the URL value with your unique collaborator or interactsh URL. Send the request. Wait 30 to 60 seconds then check your Collaborator or interactsh dashboard for incoming interactions. If you see a DNS lookup or HTTP request originating from the target server IP address, blind SSRF is confirmed beyond any doubt. The IP address in the log confirms the server's outbound network identity.

TOOLS TO USE

Burp Suite Collaborator | interactsh | manual testing

Category: Business Logic

TC-46 Malicious File Upload Leading to Remote Code Execution

WHAT IS THIS VULNERABILITY

File upload functionality is one of the highest-risk features a web application can have. When poorly implemented, it is one of the most direct paths to full server compromise. The core issue is uploading a file with a server-executable extension — .php, .asp, .aspx, or .jsp — that the web server will execute rather than serve as a static file when accessed via URL. If an attacker uploads a PHP web shell and can access it at its upload path, they have arbitrary command execution on the server. Developers often add client-side file extension checks or Content-Type header checks — but both are trivially bypassed in an intercepting proxy because they check client-controlled values.

HOW TO FIND IT — STEP BY STEP

Navigate to every file upload feature — profile picture upload, document submission, attachment in messages, import functionality. First understand what file types are accepted by uploading a legitimate PNG image. In Burp Suite, capture this successful upload request and examine the Content-Type and filename. Now create a test web shell: a PHP file containing a system call that executes a URL parameter. Attempt to upload this file directly. If rejected, try these bypass techniques one by one: change the filename extension to .php5, .phtml, .php.jpg, or .PhP with mixed case. Change the Content-Type header to image/jpeg while keeping the .php extension. Add valid JPEG magic bytes FF D8 FF E0 to the beginning of your PHP file content. Try double extensions like shell.jpg.php. If any variation succeeds and you can access the file at the upload path and execute commands through the URL parameter, Remote Code Execution is fully confirmed.

TOOLS TO USE

Burp Suite | manual file upload testing

TC-47 Purchase Price Manipulation Through Request Tampering

WHAT IS THIS VULNERABILITY

Price manipulation is a business logic vulnerability where an application trusts the price value submitted by the client in an HTTP request rather than always calculating the final price server-side from its own database. This should never happen in a properly designed application — the server should always look up the authoritative price from its own price table and use that, ignoring any value from the client. When developers build checkout flows, they sometimes include the item price in hidden form fields or JSON request bodies for convenience, and then use that submitted value in the transaction calculation without re-validating against the database. An attacker changes the price to anything they want — 1 cent, 0, or even negative.

HOW TO FIND IT — STEP BY STEP

Use Burp Suite to intercept and analyze every request during a purchase or subscription flow — from adding an item to the cart through the final payment confirmation. In each request look for any parameter containing a price, amount, total, subtotal, discount, or cost value. Look in URL parameters, JSON body, form-encoded body, and in any cookies that might carry cart state. When you find a numeric price value, send the request to Repeater. Change the price from its real value to 1. Resend and observe the response — does the server accept the modified price and create an order for 1 unit of currency? Also try setting the quantity to a negative number to see if it creates a negative charge or store credit. Each successful modification that affects what the user actually pays is a confirmed business logic vulnerability.

TOOLS TO USE

Burp Suite | manual request interception

TC-48 OTP Brute Force — No Rate Limiting

WHAT IS THIS VULNERABILITY

One-time passwords are 4 to 6 digit codes sent via SMS or email to verify identity during login, password reset, or high-value transaction confirmation. The security assumption is that the code expires quickly and an attacker cannot guess it in time. A 6-digit OTP has only one million possible values. If an attacker can make 1000 guesses per second with no rate limiting, they test every possible OTP in about 17 minutes. With a 4-digit OTP and no rate limiting, all 10,000 possibilities can be tested in seconds. The defense is rate limiting — after 3 to 5 wrong attempts, lock the OTP and require a new one. Without this protection, the entire security value of OTP verification is completely nullified.

HOW TO FIND IT — STEP BY STEP

Request an OTP to be sent to your test account. In Burp Suite, capture the OTP verification POST request — it contains the submitted OTP value as a parameter. Send this request to Intruder. Set the OTP parameter as the injection point. In the Payloads section select payload type Numbers. Set the range from 0000 to 9999 for a 4-digit OTP, or 100000 to 999999 for 6 digits. Set format to include leading zeros. Set thread count to 20. Start the attack and watch for the response code or response length that differs from all the failed attempts — this outlier is the correct OTP. If the attack completes without hitting any 429 Too Many Requests response and you identify the correct OTP, rate limiting is completely absent and the OTP mechanism provides zero real security.

TOOLS TO USE

Burp Suite Intruder | manual numeric payload testing

TC-49 Path Traversal in File Download Feature

WHAT IS THIS VULNERABILITY

Path traversal is a vulnerability where an application uses user-supplied input to construct a file path on the server and reads that file without properly restricting which directories can be accessed. If a download feature takes a filename parameter like ?file=invoice.pdf and constructs the full path as /var/www/uploads/ plus the filename, an attacker can inject path traversal sequences like ../../ to navigate outside the intended directory. The sequence ../ moves up one directory level. So ../../etc/passwd navigates from /var/www/uploads/ up two levels to / and then into /etc/passwd. In production applications these server files often contain database credentials, configuration secrets, and other extremely sensitive data.

HOW TO FIND IT — STEP BY STEP

Identify every file download or file retrieval feature in the application. Look for URL parameters like: file=, filename=, path=, doc=, download=, template=, or any parameter whose value looks like a filename or path. Start with a simple test — add ../ before the filename and see if behavior changes. Then try longer traversal sequences. For Linux servers try: ../../etc/passwd, then ../../../etc/passwd, and keep adding more sequences until one works. URL-encode the slashes if the basic version is filtered: use %2e%2e%2f or ..%2f. Try double encoding: %252e%252e%252f. For Windows servers try: ..\..\windows\system32\drivers\etc\hosts. If you successfully read /etc/passwd and see entries like root:x:0:0, path traversal is confirmed. Escalate by reading application configuration files at their typical paths for the identified technology stack.

TOOLS TO USE

Burp Suite | dotdotpwn | manual path manipulation

You have the knowledge.
Now go test responsibly.

Only test systems you have explicit written permission to test.

Report every finding you discover. Document everything carefully.

That is how we make the internet safer — one responsible disclosure at a time.

— Yamini Yadav

Security Engineer